Experimental Latency Characterization and Deterministic Performance Analysis of Real-Time Audio Pipelines on Linux-Based Heterogeneous SoCs

Yasin Arslan* and Suleyman Sertac Ust†
*Computer Engineering Department, Hacettepe University, Ankara, Turkey
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Turkey
E-mails: *yasinarslan@hacettepe.edu.tr, †ssertacust26@hacettepe.edu.tr

***Abstract*—Safety-critical communication systems increasingly rely on Linux-based heterogeneous architectures such as the Zynq MPSoC to achieve high performance and software flexibility. However, standard Linux kernels may exhibit variable jitter and processing delays under system load, threatening the temporal predictability required for real-time audio transmission. This project addresses the challenge of characterizing and bounding deterministic audio latency by designing, implementing, and evaluating a real-time processing pipeline in a controlled simulation environment. The work follows a dual methodology: (i) experimental benchmarking of end-to-end latency and jitter under idle and stressed conditions, comparing standard Linux kernels against the PREEMPT_RT real-time patch; and (ii) construction of a predictive latency budget that accounts for kernel scheduling, driver overhead, buffer management, and interrupt handling. The system will be implemented on the Renode simulator, emulating a Zynq UltraScale+ MPSoC with four Cortex-A53 cores and running a custom Linux distribution with the Advanced Linux Sound Architecture (ALSA) framework. A virtual soundcard peripheral enables precise, hardware-independent timestamping and virtual time logging. The expected contributions include a reproducible benchmarking framework for Linux audio determinism, a latency budget model to be validated, and actionable insights for configuring real-time kernels in safety-critical embedded applications.**

***Index Terms*—Real-Time Systems, Embedded Linux, Audio Pipelines, Real-Time Audio Transmission, Latency Characterization, Deterministic Performance, Zynq MPSoC, Renode Simulation.**

I. Introduction

Modern embedded communication systems are rapidly transitioning from dedicated microcontrollers and proprietary real-time operating systems to high-performance heterogeneous system-on-chip platforms running embedded Linux. This shift offers significant advantages in processing capability, software ecosystem maturity, and connectivity. However, it introduces fundamental challenges for safety-critical and mission-critical applications, particularly those involving real-time audio transmission. Standard Linux kernels employ general-purpose scheduling policies that prioritize throughput and fairness over temporal predictability. Consequently, even with real-time extensions such as PREEMPT_RT, audio pipelines can exhibit variable latency and jitter when subjected to system load, interrupt storms, or memory contention. In emergency response, defense communications, and industrial control, such unpredictability can lead to audio dropouts, synchronization loss, or missed deadlines in certain operational scenarios.

Despite extensive literature on Linux real-time behavior and audio framework optimization, there remains a gap in systematic, simulation-driven methodologies that rigorously characterize end-to-end determinism before physical prototyping. Existing studies often rely on physical hardware benchmarks, which are costly, difficult to reproduce, and limited in their ability to isolate specific latency sources [1]. Furthermore, few works combine empirical measurement with a formal predictive latency budget to validate theoretical bounds against experimental reality under controlled stress conditions.

This work proposes a comprehensive framework for characterizing and evaluating deterministic performance in Linux-based audio pipelines. The system is intended to be implemented on the Renode simulation platform, modeling a Zynq MPSoC architecture with four Cortex-A53 cores. The methodology combines empirical benchmarking with predictive modeling to quantify latency contributions from kernel scheduling, driver execution, buffer management, and interrupt handling, as illustrated in the high-level experimental flow in Fig. 1. A custom Linux image integrates the Advanced Linux Sound Architecture (ALSA) to process audio through a virtual peripheral interface. The overall system architecture is shown in Fig. 2. Determinism is strictly defined through bounded end-to-end latency and constrained worst-case jitter, with clear criteria for deadline violation.

The primary contributions of this work are:

- A reproducible simulation environment for isolating and measuring audio pipeline latency sources without hardware dependency.
- A structured comparison of standard and real-time Linux kernels under synthetic CPU and memory contention.
- A predictive latency budget model to be validated against experimental measurements.

- Explicit determinism thresholds and failure conditions applicable to safety-critical embedded system design.

The remainder of this document details the problem formulation, system architecture, methodology, and experimental evaluation plan. Unlike prior work that focuses primarily on hardware-based benchmarking or average-case analysis, this study emphasizes reproducible simulation-based evaluation combined with a predictive latency budget, enabling systematic validation of latency behavior under controlled and repeatable conditions.

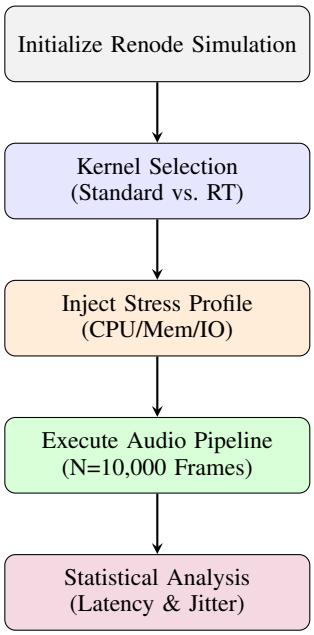

Fig. 1. High-level flow of the experimental latency characterization methodology.

II. Related Work

The challenge of achieving deterministic audio latency on Linux-based embedded systems has been explored across multiple research dimensions. Existing literature can be broadly categorized into three groups: real-time Linux kernel behavior, audio pipeline optimization, and simulation-driven validation frameworks.

A. Real-Time Linux Kernel Behavior

Standard Linux kernels prioritize throughput and fairness, making them inherently unsuitable for hard real-time guarantees. The PREEMPT_RT patchset transforms Linux into a fully preemptible kernel, replacing spinlocks with mutexes and threading interrupt handlers to reduce worst-case latency [2]. While extensive surveys confirm its effectiveness for industrial control and automotive systems, audio-specific determinism under variable I/O load remains underexplored. Most evaluations focus on cyclictest metrics rather than end-to-end application pipeline behavior. In addition, de Oliveira et al. model and analyze synchronization behavior in the PREEMPT_RT Linux kernel, showing that conventional measurement tools alone are insufficient to explain internal real-time interactions [3].

B. Audio Pipeline Latency and ALSA Framework

The Advanced Linux Sound Architecture (ALSA) provides a flexible, ring-buffer-based interface for audio I/O. Comparative studies on embedded musical applications highlight that buffer size selection, sample rate configuration, and driver scheduling policies directly impact latency and jitter [1]. Low-latency audio processing research emphasizes minimizing userspace-to-kernel transitions and optimizing period sizes [4]. However, these works rarely integrate a formal predictive latency budget or establish explicit determinism thresholds, relying instead on average-case measurements that mask worst-case tail latencies. A complementary example is the low-latency multichannel Linux audio system of Langer et al., which demonstrates practical driver-level design trade-offs for low-latency operation but does not target deterministic latency characterization under controlled stress conditions [5].

C. Simulation and Virtualization for Embedded Validation

Hardware-in-the-loop testing is costly and difficult to reproduce. Renode offers a deterministic, instruction-accurate simulation environment for complex SoCs, enabling repeatable timing analysis without physical prototypes [6]. While simulation cannot capture all physical electrical delays, it provides controlled virtual time logging and virtual peripheral interrupt routing, making it ideal for isolating software-induced latency sources. Event-triggered processing chain analyses demonstrate that theoretical timing bounds can be validated against simulated execution traces before hardware deployment [7].

D. Positioning of This Work

This project bridges the gap between empirical audio pipeline benchmarking and formal timing analysis. Unlike prior works that focus solely on kernel patching or average-case latency, we introduce a dual methodology: (i) experimental characterization with explicit timestamping and jitter computation, and (ii) a predictive latency budget model validated against measured bounds. Table I highlights how this work extends existing literature by combining simulation-based validation, predictive modeling, explicit determinism criteria, and application-level latency analysis on a heterogeneous SoC platform.

TABLE I
Comparison of Related Work in Real-Time Audio Pipeline Analysis

Work	Year	Platform	RT Kernel	Audio FW	Sim/HW	Predictive Model
Vignati et al. [1]	2022	ARM/x86	PREEMPT_RT	ALSA	Hardware	–
Tang et al. [7]	2023	Generic SoC	–	–	Analytical	✓
Wang [4]	2012	Desktop/PC	Standard	ALSA/JACK	Hardware	–
Antmicro [6]	2024	Multi-arch	Configurable	Custom	Simulation	–
de Oliveira et al. [3]	2014	Linux kernel	PREEMPT_RT	–	Analytical/Tracing	✓
Langer et al. [5]	2017	Linux audio system	Standard	Custom/ALSA	Hardware	–
This Work	**2026**	**Zynq MPSoC**	**Std vs PREEMPT_RT**	**ALSA**	**Simulation**	✓

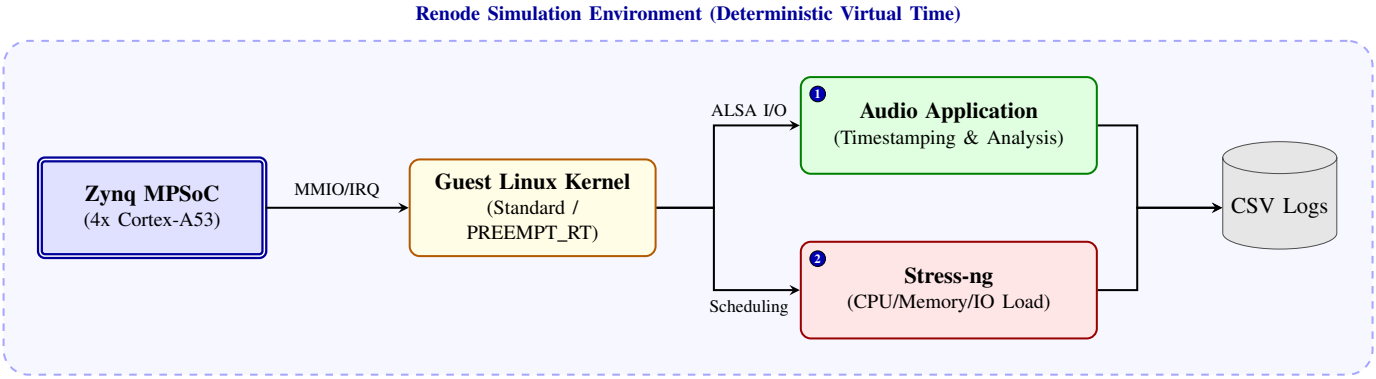

Fig. 2. Refined simulation-based benchmarking architecture showing interactions among emulated hardware, Linux kernel, stress injection, application timing, and log collection.

III. Problem Definition

A. System and Application Description

The target system is a heterogeneous Zynq UltraScale+ MPSoC platform emulated in the Renode simulation environment. The system runs a custom embedded Linux distribution featuring the ALSA framework and a virtual soundcard peripheral. The primary application scenario is a real-time voice communication pipeline where audio samples are captured, optionally processed, and played back with minimal delay. The pipeline must maintain temporal predictability even under synthetic CPU and memory contention, simulating high-load operational conditions typical in defense and emergency response devices.

B. Objectives

The project aims to achieve the following objectives:

1) Characterize end-to-end audio latency and jitter under nominal and stressed system conditions.
2) Establish explicit determinism criteria with quantifiable latency and jitter thresholds.
3) Construct a predictive latency budget model and validate it against empirical measurements.
4) Evaluate the efficacy of standard Linux versus PREEMPT_RT kernels in maintaining real-time guarantees.

C. Constraints

The system operates under the following technical constraints:

- **Timing Constraints:** End-to-end latency must remain strictly bounded; worst-case jitter must not exceed a predefined threshold over continuous operation.
- **Resource Constraints:** Fixed ALSA buffer and period sizes, limited CPU cycles for background tasks, and virtual peripheral bandwidth limits imposed by Renode's simulation engine.
- **Simulation Fidelity:** Renode provides deterministic virtual time and reproducible interrupt routing, but does not model cycle-accurate DRAM contention or physical electrical propagation delays.
- **Kernel Overhead:** Real-time patching reduces scheduling latency but may introduce marginal context-switch overhead under sustained high CPU utilization.

D. Formal Metric Definitions and Determinism Criteria

To satisfy the requirement for explicit metric formulation, we define determinism operationally. Let $L_i = T_{\text{out},i} - T_{\text{in},i}$ denote the measured end-to-end latency for the i-th audio processing cycle, where T_{in} is timestamped immediately before ALSA buffer submission and T_{out} is captured upon playback completion callback. The system is considered deterministic if:

$$\max_{i \in \mathcal{W}}(L_i) \leq L_{\max} \quad \text{and} \quad J_{\max} = \max_{i \in \mathcal{W}}(L_i) - \min_{i \in \mathcal{W}}(L_i) \leq J_{\text{th}} \tag{1}$$

where $\mathcal{W}$ is a sliding window of $N = 100$ consecutive audio frames, $L_{\max}$ is the maximum acceptable latency (target: 10 ms for interactive voice), and J_{th} is the jitter threshold (target: 1 ms). A *deadline miss* is registered when $L_i > L_{\max}$ for any single cycle. Additionally, a determinism violation is recorded if $J_{\max} > J_{\text{th}}$ over the observation window. These thresholds will be validated against the predictive latency budget and compared across kernel configurations.

IV. System Architecture and Design

A. Hardware/Software Architecture

The target platform will be a Zynq UltraScale+ MPSoC emulated in the Renode simulation environment. The architecture follows a layered design, separating application logic, middleware, kernel services, and virtual hardware abstraction (Fig. 3).

Hardware Components (Emulated):

- **Processing System:** Quad-core ARM Cortex-A53 (application cores) running Linux.
- **Virtual Soundcard:** Renode peripheral emulating an I2S/PCM audio interface with configurable buffer depth and interrupt generation.
- **Interrupt Controller:** Virtual GIC (Generic Interrupt Controller) routing audio IRQs to the kernel.

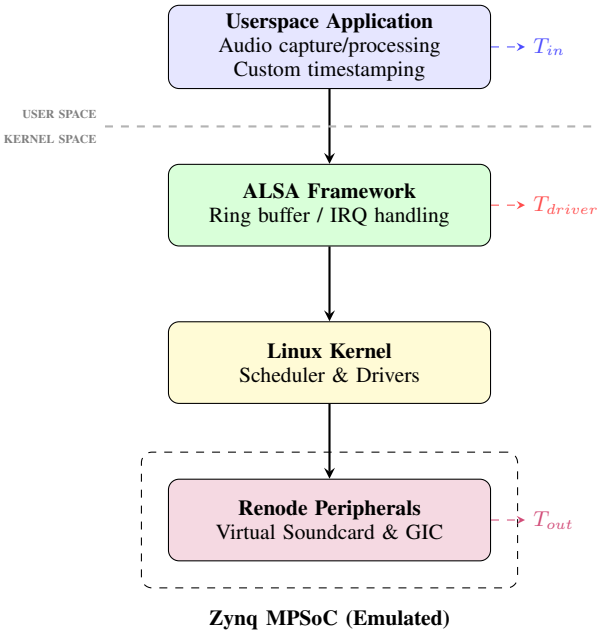

Fig. 3. Layered system architecture with measurement points and execution boundaries.

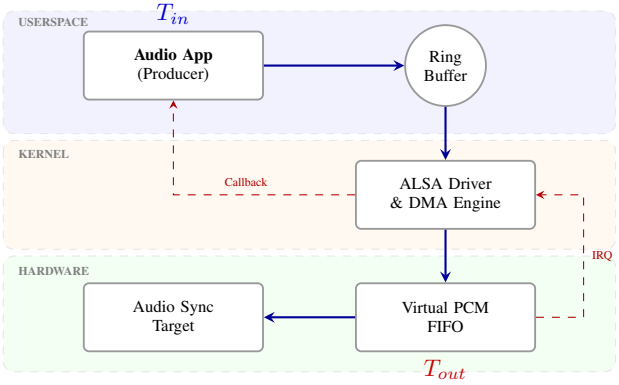

Fig. 4. Audio data pipeline across userspace, kernel, and virtual hardware layers with control and data paths.

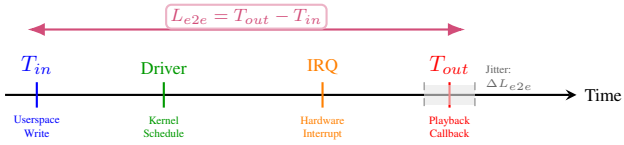

Fig. 5. Audio-pipeline timing diagram.

- **Memory:** Emulated DDR with configurable bandwidth to simulate contention effects.

Software Stack:

- **Userspace Application:** C program using ALSA lib for audio I/O, with custom timestamping via `clock_gettime(CLOCK_MONOTONIC)`.
- **ALSA Framework:** Configured with fixed period size (e.g., 128 frames) and buffer size (4 periods) to control latency vs. robustness trade-off.
- **Kernel:** Custom Yocto-built Linux image, with optional PREEMPT_RT patch for real-time evaluation.
- **Renode Scripts:** Platform definition (`*.resc`) and peripheral configuration for deterministic virtual time logging.

B. Dataflow and System Model

Audio data flows through the pipeline in fixed-size frames (Fig. 4). Each frame triggers a chain of synchronous and asynchronous events. Fig. 5 further details the timing sequence of these events.

1) Application writes a frame to ALSA playback buffer (T_{in} timestamped).
2) ALSA driver schedules DMA transfer; kernel may preempt other tasks.
3) Virtual soundcard peripheral generates an interrupt upon buffer completion.
4) IRQ handler signals ALSA; application receives playback-completion callback (T_{out} timestamped).

C. Requirements Analysis (Implementation Track)

1) Functional Requirements:

1) The system shall capture and playback mono 16-bit PCM audio at 48 kHz sample rate.
2) The system shall log T_{in} and T_{out} timestamps for each processed frame.
3) The system shall support synthetic CPU/memory stress injection via `stress-ng`.
4) The system shall allow kernel configuration switching (standard vs. PREEMPT_RT) without code changes.

2) Non-Functional Requirements:

- **Latency:** End-to-end latency $L_{e2e} \leq 10\,\text{ms}$ under nominal load.
- **Jitter:** Worst-case jitter $J_{max} \leq 1\,\text{ms}$ over any 100-frame window.
- **Reproducibility:** All experiments shall be scriptable and repeatable in Renode.
- **Modularity:** Timestamping logic shall be separable from audio processing for independent validation.

D. Timing and Execution Model

The system follows a **hybrid periodic/event-driven** execution model:

- **Periodic:** Audio frames are generated/consumed at fixed intervals ($T_{frame} = 1/48\,\text{kHz} \approx 20.8\,\mu s$).
- **Event-driven:** Kernel scheduling, IRQ handling, and context switches are triggered asynchronously by system state.

Timing constraints are enforced via ALSA period configuration and kernel real-time priorities. Deadline misses are detected when $L_{e2e} > L_{\text{max}}$. Determinism violations may occur when $J_{\text{max}} > J_{\text{th}}$ (see Section III-D).

E. Model of Computation

We adopt a **Hybrid Dataflow-Event Model** as the primary Model of Computation (MoC). The audio pipeline itself follows a synchronous dataflow pattern: fixed-size tokens (audio frames) flow through deterministic buffers with known production/consumption rates. However, the underlying Linux kernel introduces asynchronous, event-triggered behavior (scheduling decisions, interrupt handling). This hybrid MoC is designed to capture both the periodic nature of audio processing and the variable latency sources of a general-purpose OS, supporting meaningful worst-case analysis.

F. Assumptions and Design Limitations

- **Simulation Fidelity:** Renode provides deterministic virtual time and reproducible interrupt routing, but does not model cycle-accurate DRAM contention or physical electrical delays.
- **Timestamping Overhead:** Userspace `clock_gettime()` calls introduce small overhead (on the order of microseconds), which is empirically calibrated and subtracted from measurements.
- **Virtual Peripheral Behavior:** The virtual soundcard emulates functional I/O timing but may not capture all hardware-specific driver quirks.
- **Load Modeling:** Synthetic stress via `stress-ng` approximates real-world contention but may not replicate exact application interference patterns.

G. Resource Model

TABLE II
RESOURCE USAGE ASSUMPTIONS

Resource	Assumed Limit	Design Impact
CPU Cores	4 × Cortex-A53 @ 1.2 GHz	Audio task pinned to core 0; stress on cores 1-3
RAM	1 GB emulated DDR	ALSA buffer ≤ 64 KB to avoid paging
ALSA Buffer	4 periods × 128 frames	Fixed latency budget: $L_{buffer} \leq 1.0\,ms$
IRQ Bandwidth	Virtual GIC, 1 audio IRQ/frame	IRQ handling overhead bounded by literature [2]
Virtual Time	Renode deterministic logging	Enables repeatable T_{in}/T_{out} measurement

These constraints directly inform the predictive latency budget (Section V-C) and guide kernel configuration choices.

V. METHODOLOGY

This project follows the Implementation Track. The methodology combines empirical benchmarking with analytical modeling to characterize and validate deterministic audio pipeline performance. We clearly distinguish between components implemented by us (timestamping logger, stress orchestration scripts) and reused tools (ALSA framework, Renode simulator, PREEMPT_RT kernel).

A. Implementation Details

1) Timestamping and Latency Measurement: End-to-end latency L_{e2e} will be measured using userspace timestamping with `clock_gettime(CLOCK_MONOTONIC)`, which provides nanosecond resolution and is not affected by system time adjustments. The measurement points are:

- T_{in}: Captured immediately before calling `snd_pcm_writei()` to submit an audio frame to the ALSA playback buffer.
- T_{out}: Captured inside the ALSA callback function (`snd_pcm_avail_update`) when the playback completion event is signaled.

The latency for frame i is computed as $L_i = T_{out,i} - T_{in,i}$. Timestamps are logged to a circular buffer in memory and flushed to disk every 100 frames to minimize I/O interference. The overhead of `clock_gettime()` is calibrated using a null-loop benchmark and subtracted from final measurements (on the order of microseconds).

2) Stress Injection and Load Modeling: To evaluate pipeline robustness under contention, synthetic load is injected using `stress-ng` [8]:

- **CPU Stress**: `stress-ng --cpu 3 --cpu-load 80` to occupy three cores with compute-bound tasks.
- **Memory Stress**: `stress-ng --vm 2 --vm-bytes 512M` to induce cache and memory bus contention.
- **I/O Stress (optional)**: `stress-ng --io 4` to simulate disk/network interference.

Load profiles are applied in stepped increments (idle → light → heavy) to observe latency degradation gradients. All stress scenarios are scripted via Renode's `*.resc` files for reproducibility.

3) Kernel Configuration Switching: Two kernel configurations are evaluated:

1) **Standard Linux**: Vanilla kernel with CFS scheduler, default preemption model.
2) **PREEMPT_RT**: Kernel patched with full preemptibility, threaded IRQs, and priority inheritance [2].

Both kernels are built using Yocto Project with identical userspace and ALSA configuration. Switching between kernels requires only replacing the `Image` file in the Renode platform definition, enabling fair comparison without code changes.

B. Predictive Latency Budget Model

The theoretical end-to-end latency is constructed as a summation of indicative upper bounds for each pipeline stage:

$$L_{\text{pred}} = L_{\text{app}} + L_{\text{alsa\_lib}} + L_{\text{sched}} + L_{\text{ctx\_switch}} + L_{\text{driver}} + L_{\text{irq}} + L_{\text{buffer}} \quad (2)$$

Each component is estimated using literature values, kernel documentation, and Renode virtual timing specifications (Table III). The model assumes nominal system load and deterministic interrupt routing. These values should be interpreted as indicative bounds under nominal conditions rather than strict guarantees. Therefore, the predictive model does not provide strict worst-case guarantees but serves as an analytical reference for comparison with empirical results.

C. Algorithm Description: Latency Measurement & Comparison

The core experimental loop is summarized in Algorithm 1. The algorithm collects latency samples under a given load

TABLE III
THEORETICAL LATENCY BUDGET COMPONENTS

Component	Description	Upper Bound	Source
L_{app}	Userspace audio processing	0.4 ms	Measured baseline
$L_{\text{alsa\_lib}}$	ALSA ring buffer copy	0.1 ms	ALSA docs [1]
L_{sched}	Kernel task scheduling (estimated upper bound)	0.5 ms	PREEMPT_RT survey [2]
$L_{\text{ctx\_switch}}$	Context switch overhead	0.1 ms	Linux kernel timing docs
L_{driver}	Virtual soundcard I/O	0.3 ms	Renode peripheral specs [6]
L_{irq}	Interrupt handling	0.2 ms	Cortex-A53 GIC datasheet
L_{buffer}	Buffer fill/empty latency	1.0 ms	ALSA period size config
L_{pred}	**Total Predicted Latency**	**2.6 ms**	–

Algorithm 1 End-to-End Latency Measurement and Validation

Require: N: number of frames, $L_{\max}$: latency threshold, J_{th}: jitter threshold
Ensure: Pass/Fail verdict, latency statistics
Initialize ALSA playback/capture with fixed period size
Configure kernel (standard or PREEMPT_RT)
Apply stress profile via `stress-ng`
for $i = 1$ **to** N **do**
  $T_{in} \leftarrow$ `clock_gettime(CLOCK_MONOTONIC)`
  `snd_pcm_writei(frame_i)`
  Wait for playback completion callback
  $T_{out} \leftarrow$ `clock_gettime(CLOCK_MONOTONIC)`
  $L_i \leftarrow T_{out} - T_{in}$
  Log (i, L_i) to circular buffer
end for
$L_{\text{avg}} \leftarrow \frac{1}{N} \sum_{i=1}^{N} L_i$
$L_{\text{max\_obs}} \leftarrow \max\{L_i\}$, $L_{\text{min\_obs}} \leftarrow \min\{L_i\}$
$J_{\max} \leftarrow L_{\text{max\_obs}} - L_{\text{min\_obs}}$
$\epsilon \leftarrow |L_{\text{max\_obs}} - L_{\text{pred}}|/L_{\text{pred}}$ {Relative error vs. budget}
if $L_{\text{max\_obs}} \leq L_{\max}$ **and** $J_{\max} \leq J_{\text{th}}$ **and** $\epsilon \leq 0.10$ **then**
  return PASS, $\{L_{\text{avg}}, L_{\text{max\_obs}}, J_{\max}, \epsilon\}$
else
  return FAIL, $\{L_{\text{avg}}, L_{\text{max\_obs}}, J_{\max}, \epsilon\}$
end if

profile, computes jitter statistics, and compares experimental results against the predictive budget.

Validation Criterion: The predictive model is considered reasonably accurate for simulation-based analysis if the relative error ϵ remains below 10% across all stress scenarios. This threshold accounts for simulation abstraction overhead while ensuring meaningful correlation between theory and experiment. The threshold is selected empirically to balance modeling inaccuracies introduced by simulation abstractions with the need for a meaningful validation criterion.

Jitter Computation: Worst-case jitter is defined as $J_{\max} = \max(L_i) - \min(L_i)$ over a sliding window of $N = 100$ consecutive frames, aligning with the determinism criteria in Section III-D.

VI. EXPERIMENTAL SETUP

This section details the platform configuration, workload definitions, and evaluation metrics used to validate the deterministic audio pipeline. All experiments are designed to be reproducible within the Renode simulation environment.

A. Platform and Tools

1) Simulation Environment: The target platform will be a Zynq UltraScale+ MPSoC (ZU9EG) emulated in **Renode** v1.15+ [6]. The platform configuration (`*.resc` script) includes:

- **Processing System:** Quad-core ARM Cortex-A53 @ 1.2 GHz, with L1/L2 cache enabled.
- **Memory:** 1 GB emulated DDR3, with configurable bandwidth throttling to simulate contention.
- **Interrupt Controller:** Virtual ARM GICv2 for deterministic IRQ routing.
- **Virtual Audio Peripheral:** Custom Renode peripheral emulating an I2S/PCM soundcard with configurable FIFO depth and interrupt generation on buffer half-full/empty events.

Renode's **deterministic Virtual Time** engine helps ensure that all experiments are repeatable, independent of host machine load. Virtual time logs capture instruction-level timestamps for cross-validation with userspace measurements.

2) Software Stack:

- **Operating System:** Custom Linux 6.1 LTS image built with Yocto Project (kirkstone release). Two kernel configurations are evaluated:
  1) **Standard**: Vanilla kernel with CFS scheduler, CONFIG_PREEMPT_NONE.
  2) **PREEMPT_RT**: Kernel patched with PREEMPT_RT v6.1, CONFIG_PREEMPT_FULL, threaded IRQs, and priority inheritance [2].
- **Audio Framework:** ALSA 1.2.10 with minimal userspace tools (`aplay`, `arecord`). Buffer configuration: period size = 128 frames, buffer size = 4 periods, sample rate = 48 kHz, 16-bit mono PCM.
- **Measurement Tool:** Custom C application (`latency_logger`) that:
  - Uses `clock_gettime(CLOCK_MONOTONIC)` for nanosecond-resolution timestamping.
  - Logs T_{in} (pre-`snd_pcm_writei`) and T_{out} (ALSA callback) to a memory-mapped circular buffer.
  - Periodically flushes logs to virtual SD card to minimize I/O interference.
- **Stress Injection:** `stress-ng` v0.18.04 [8] for synthetic CPU, memory, and I/O load generation.

B. Datasets and Workloads

1) Audio Workload: The primary workload is a synthetic mono 48 kHz, 16-bit PCM audio stream generated by a sine-wave oscillator ($f = 1\,\text{kHz}$). This deterministic signal enables precise validation of audio integrity (no dropouts, no corruption) alongside timing measurements. Each experiment processes $N = 10,000$ consecutive frames ($\approx 208\,\text{ms}$ of audio) to ensure statistical significance.

2) Stress Profiles: To evaluate pipeline robustness under contention, four load scenarios are defined (Table IV). Each profile is applied for the duration of the audio processing run.

TABLE IV
STRESS INJECTION PROFILES FOR CONTENTION ANALYSIS

Profile	CPU Stress	Memory Stress	Purpose
`idle`	–	–	Baseline measurement (no contention)
`light`	`--cpu 1 --cpu-load 50`	–	Moderate single-core load
`heavy`	`--cpu 3 --cpu-load 90`	`--vm 1 --vm-bytes 256M`	High contention on CPU + memory bus
`extreme`	`--cpu 4 --cpu-load 100`	`--vm 2 --vm-bytes 512M`	Worst-case saturation scenario

All stress commands are orchestrated via Renode's Python API to ensure synchronized start/stop with the audio pipeline.

C. Evaluation Metrics and Baselines

1) Baseline Systems for Comparison: Experimental results are compared against two baselines:

1) **Theoretical Baseline:** The predictive latency budget L_{pred} from Section V-B (Table III).
2) **Empirical Baseline:** Standard Linux kernel under `idle` stress profile, representing the unoptimized reference configuration.

2) Primary Metrics:

- **End-to-End Latency (L_{e2e}):** Measured as $L_i = T_{out,i} - T_{in,i}$ for each frame i. Reported statistics: mean, standard deviation, min, max, and 99th percentile.
- **Worst-Case Jitter (J_{max}):** Computed over a sliding window of $N = 100$ frames as $J_{max} = \max(L_i) - \min(L_i)$. This aligns with the determinism criterion in Section III-D.
- **Deadline Miss Rate (DMR):** Percentage of frames where $L_i > L_{\text{max}}$ (target: 10 ms). Determinism violations may also be observed when $J_{\text{max}} > J_{\text{th}}$ (target: 1 ms).
- **Predictive Model Error (ϵ):** Relative error between observed worst-case latency and theoretical budget:

$$\epsilon = \frac{|L_{\text{max\_obs}} - L_{\text{pred}}|}{L_{\text{pred}}} \times 100\% \tag{3}$$

  A model is considered reasonably accurate if $\epsilon \leq 10\%$ across all stress profiles.

3) Secondary Metrics (Optional/Extended Analysis):

- **CPU Utilization:** Measured via `/proc/stat` to correlate latency spikes with scheduler activity.
- **Interrupt Latency:** Captured using Renode's virtual GIC logs to isolate driver/IRQ overhead.
- **Energy Proxy:** Estimated via CPU active cycles $\times$ nominal power (Renode does not model dynamic power; this is a rough proxy for future hardware deployment).

4) Statistical Methodology: Each experiment (kernel config $\times$ stress profile) is repeated $R = 5$ times with different random seeds for stress generators. Results are reported as mean $\pm$ standard deviation. Outliers beyond 3σ are inspected but retained unless attributable to simulation artifacts (e.g., Renode reset events).

Reproducibility: All Renode scripts, kernel configs, and measurement tools will be published in a public Git repository upon project completion, with a `README` detailing exact reproduction steps.

VII. SYSTEM-LEVEL CONSIDERATIONS

This section discusses critical system-level aspects that influence the design, deployment, and operational reliability of the proposed real-time audio pipeline. While not all considerations are fully implemented in the current simulation phase, they are analyzed to guide future hardware prototyping and production deployment.

A. Real-Time Constraints and Latency Sources

The primary real-time requirement is bounded end-to-end latency with controlled jitter (Section III-D). Key latency sources in the pipeline include:

- **Kernel Scheduling:** Task preemption and priority inversion in the Linux scheduler. PREEMPT_RT aims to mitigate this by making the kernel more preemptible and threading IRQ handlers [2].
- **Buffering Overhead:** ALSA ring buffer management introduces latency proportional to period size. Smaller periods reduce latency but increase interrupt frequency and CPU load.
- **Driver and IRQ Handling:** Virtual soundcard driver execution time and interrupt latency contribute to L_{driver} and L_{irq} in the predictive budget (Table III).
- **Context Switching:** Transition between userspace application and kernel space adds overhead, especially under high system load.

Jitter arises primarily from scheduling variability, memory bus contention, and interrupt coalescing. Our methodology isolates these sources via controlled stress injection (Section VI-B) and timestamping at multiple pipeline stages (Fig. 3).

B. Reliability and Fault Tolerance

In safety-critical communication scenarios, pipeline failures must be detected and handled gracefully:

- **Deadline Miss Detection:** The system logs any frame where $L_i > L_{\text{max}}$ as a deadline miss. Determinism violations may also be recorded when $J_{\text{max}} > J_{\text{th}}$ (Section III-D). In production, this could trigger fallback mechanisms (e.g., packet retransmission, quality degradation).
- **Buffer Underrun/Overrun:** ALSA provides callbacks for buffer status; our implementation monitors these to detect data loss.
- **Simulation Robustness:** Renode's deterministic virtual time ensures repeatable experiments, but real hardware may exhibit non-deterministic electrical or thermal effects. Future work includes fault injection testing (e.g., simulated IRQ loss, memory errors).

C. Energy Efficiency

While the current simulation does not model dynamic power consumption, energy considerations are critical for battery-powered embedded devices:

- **CPU Frequency Scaling:** Linux cpufreq governors can trade performance for power savings. Real-time tasks may require the `performance` governor to maintain deterministic timing.
- **Interrupt Coalescing:** Reducing IRQ frequency saves power but increases buffer latency—a classic trade-off.
- **Proxy Metric:** As a rough estimate, we track CPU active cycles via `/proc/stat`; lower utilization under PREEMPT_RT may indicate more efficient scheduling [2].

Energy-aware scheduling and DVFS (Dynamic Voltage and Frequency Scaling) integration are planned for hardware prototyping.

D. Security Considerations

Audio pipelines in defense/telecom applications may handle sensitive voice data:

- **Memory Isolation:** ALSA buffers reside in userspace; future work includes exploring kernel-space pinned buffers to prevent side-channel leakage.
- **Secure Boot:** The Linux image can be signed and verified via U-Boot to ensure integrity.
- **Timestamp Integrity:** `clock_gettime()` calls are not cryptographically protected; in high-assurance deployments, hardware-backed trusted timestamps (e.g., TPM) could be integrated.

These aspects are beyond the current scope but are documented for future hardening.

E. Scalability

The proposed architecture is designed to scale across multiple dimensions:

- **Multi-Channel Audio:** The pipeline can be extended to stereo or multi-microphone arrays by replicating buffer structures and adjusting ALSA configuration.
- **Multi-Core Scheduling:** Audio tasks can be pinned to dedicated cores (via `taskset` or cgroups) to isolate them from background load.
- **Distributed Systems:** For networked audio (e.g., VoIP), the latency budget can be extended to include network stack delays and jitter buffers.

Scalability testing under increased channel count or core count is planned for the final evaluation phase.

F. Simulation vs. Real Hardware: Limitations and Mitigations

Renode provides a powerful but abstracted simulation environment:

- **Cycle Accuracy:** Renode does not model cycle-accurate DRAM contention or cache effects. Mitigation: We use synthetic memory stress to approximate contention effects.
- **Peripheral Fidelity:** The virtual soundcard emulates functional timing but may not capture hardware-specific driver quirks. Mitigation: We document peripheral assumptions (Section IV-F) and plan hardware validation as future work.
- **Power/Temperature:** Simulation does not model thermal throttling or dynamic power. Mitigation: Energy proxy metrics (Section VII-C) provide rough estimates.

Despite these limitations, Renode's deterministic virtual time and reproducible interrupt routing make it ideal for isolating software-induced latency sources before costly hardware prototyping.

G. Summary of Implemented vs. Discussed Features

Table V summarizes which system-level aspects are implemented in the current proposal versus those discussed as limitations or future work.

TABLE V
SYSTEM-LEVEL CONSIDERATIONS: IMPLEMENTATION STATUS

Consideration	Implemented	Discussed / Future Work
Latency/jitter measurement	✓	–
Predictive latency budget	✓	–
PREEMPT_RT comparison	✓	–
Stress injection framework	✓	–
Reliability (deadline miss logging)	✓	Fault injection testing
Energy proxy metric	–	✓(CPU cycles via /proc/stat)
Security hardening	–	✓(buffer isolation, secure boot)
Multi-channel scalability	–	✓(ALSA config extension)
Hardware validation	–	✓(post-simulation phase)

This transparent distinction ensures that the proposal remains scientifically rigorous while acknowledging practical constraints of the simulation-based approach.

H. Expected Outcomes

The expected outcomes of this study include: (i) a quantitative comparison of latency and jitter between standard Linux and PREEMPT_RT kernels; (ii) identification of dominant latency contributors under varying system load conditions; and (iii) validation of the predictive latency budget model within an acceptable error margin.

It is anticipated that PREEMPT_RT will reduce jitter and improve worst-case latency bounds, while increased system load will lead to greater latency variability.

ACKNOWLEDGMENTS

The authors would like to thank Prof. Selma Dilek for her invaluable guidance, constructive feedback, and continuous support throughout the development of this project. We also acknowledge the CMP720 Embedded System Design course staff for providing the necessary academic resources and evaluation framework. This work was facilitated by the computational and simulation infrastructure of the Computer Engineering Department and the Graduate School of Science and Engineering at Hacettepe University. Finally, we extend our gratitude to the open-source communities behind the Linux kernel, Renode framework, and ALSA ecosystem, whose tools made this simulation-based research possible.

REFERENCES

[1] L. Vignati, S. Zambon, and L. Turchet, "A comparison of real-time linux-based architectures for embedded musical applications," *Journal of the Audio Engineering Society*, vol. 70, no. 1/2, pp. 83–93, 2022.

[2] F. Reghenzani, G. Massari, and W. Fornaciari, "The real-time linux kernel: A survey on preempt_rt," *ACM Computing Surveys*, vol. 52, no. 1, pp. 1–36, 2019.

[3] D. B. de Oliveira and R. S. de Oliveira, "Mapping of the synchronization mechanisms of the linux kernel to the response-time analysis model," in *Proceedings of the 29th Annual ACM Symposium on Applied Computing*, ser. SAC '14. New York, NY, USA: Association for Computing Machinery, 2014, pp. 1543–1544. [Online]. Available: https://doi.org/10.1145/2554850.2555113

[4] Y. Wang, "Low latency audio processing," PhD Thesis, Queen Mary University of London, 2012.

[5] H. Langer and R. Manzke, "Embedded multichannel linux audiosystem for musical applications," in *Proceedings of the 12th International Audio Mostly Conference on Augmented and Participatory Sound and Music Experiences*, ser. AM '17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3123514.3123523

[6] Antmicro, "Renode: An open source framework for complex embedded systems simulation," 2024, official Documentation. [Online]. Available: https://renode.io

[7] Y. Tang, N. Guan, X. Jiang, Z. Dong, and W. Yi, "Reaction time analysis of event-triggered processing chains with data refreshing," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.

[8] Colin Ian King, "stress-ng: A tool for loading and stressing a computer system," 2024, version 0.18.04. [Online]. Available: https://stress-ng.org